AI ENABLEMENT

Internship Project Brief

Choose Your Challenge · Build · Showcase

Welcome to the AI Enablement Internship! This brief outlines two real-world AI agent prototype challenges. Read both tasks carefully and select the **ONE** that excites you most.

TASK 1		TASK 2
HR Resume & LinkedIn Shortlisting Agent	**VS**	**Finance Credit Follow-Up Email Agent**

Mandatory Disclosure: Your submission must include documentation of the LLM model chosen, the agent framework used, and a dedicated section on how you have mitigated security risks (prompt injection, data privacy, credential handling, etc.).

TASK 1 · HR Resume & LinkedIn Shortlisting Agent

Overview

Design and build an AI agent prototype that assists an HR team in evaluating candidates efficiently. The agent ingests a Job Description (JD) along with a batch of resumes (PDF/DOCX) and/or LinkedIn profile data, then produces a ranked shortlist with a transparent scoring rubric explaining every score.

Business Problem

HR teams routinely screen hundreds of applications per role, leading to fatigue, inconsistency, and unconscious bias. A well-designed AI agent can standardise evaluation, highlight skill gaps, and surface the best-fit candidates faster — all while keeping a human in the loop for final decisions.

Core Features to Build

- **JD Parser:** Extract key requirements — skills, experience, qualifications — from the Job Description.
- **Resume/LinkedIn Ingestion:** Accept PDF/DOCX resumes OR LinkedIn profile JSON/scrape data as input.
- **Semantic Matching Engine:** Compare each candidate profile against the JD using embeddings or LLM reasoning.
- **Scoring Rubric:** Produce a structured score for each candidate across at least 5 dimensions (see rubric table below).
- **Shortlist Report:** Output a ranked table (PDF/HTML/JSON) with scores, justifications, and a hire/no-hire recommendation.
- **Human-in-the-Loop Hook:** Allow HR to override or flag a candidate score with a reason.

Scoring Rubric (Mandatory Output Format)

Dimension	Weight	0 – Poor	5 – Average	10 – Excellent
Skills Match	30 %	< 30 % skills match	50–70 % skills match	> 85 % skills match
Experience Relevance	25 %	Unrelated domain	Adjacent domain	Exact domain & seniority
Education & Certs	15 %	Does not meet minimum	Meets minimum	Exceeds + extra certs
Project / Portfolio	20 %	No evidence	1–2 generic projects	Strong relevant portfolio
Communication Quality	10 %	Poor structure/grammar	Adequate clarity	Crisp, structured, impactful

> The agent must print dimension-level scores, the weighted total, and a one-line justification per dimension.

Sample Agent Flow

1. Input	HR uploads JD + resume files / LinkedIn URLs
2. Parse	LLM extracts structured requirements from JD
3. Profile	Agent parses each resume / LinkedIn profile into structured fields
4. Score	Agent computes rubric scores using LLM + optional embedding similarity
5. Rank	Candidates sorted by total score descending
6. Report	Shortlist report generated (PDF/HTML/JSON) with rubric breakdown
7. Override	HR can adjust scores; agent logs the change and reason

Suggested Tech Stack

Layer	Suggested Options
LLM	GPT-4o / Claude 3.5 Sonnet / Gemini 1.5 Pro / Mistral Large
Agent Framework	LangChain Agents / LlamaIndex / CrewAI / AutoGen / LangGraph
Embeddings	OpenAI text-embedding-3-small / BGE / Sentence-Transformers
Resume Parse	PyMuPDF / pdfplumber / python-docx + LLM extraction
LinkedIn	RapidAPI LinkedIn scraper OR manually exported JSON
Output	ReportLab PDF / Jinja2 HTML / JSON API
UI (Optional)	Streamlit / Gradio / FastAPI + React

TASK 2 · Finance Credit Follow-Up Email Agent

Overview

Build an AI agent prototype that supports the Finance team by automatically generating and (optionally) sending follow-up emails for pending credit/invoice payments. The agent must vary the tone and urgency of each follow-up based on the number of reminders already sent — starting politely and escalating progressively.

Business Problem

Finance teams spend significant time chasing overdue payments. Manual follow-ups are inconsistent in tone and timing. An AI agent can automate this workflow, maintain professional communication, escalate appropriately without human intervention, and log every interaction for audit — reducing DSO (Days Sales Outstanding) while preserving client relationships.

Core Features to Build

- **Data Ingestion:** Read pending credit records from CSV / Excel / mock database (invoice no., client, amount, due date, contact email, follow-up count).
- **Tone Escalation Engine:** Generate a different-toned email for each follow-up stage (see escalation matrix below).
- **Email Generation:** LLM generates a personalised, professional email for each debtor at the correct tone level.
- **Trigger Logic:** Agent identifies all overdue records and dispatches the appropriate follow-up stage automatically.
- **Send / Mock Send:** Integrate with SMTP / SendGrid / Mailgun — OR simulate sending with a dry-run log.
- **Audit Trail:** Every generated email is logged with timestamp, tone used, invoice details, and send status.
- **Escalation Cap:** After Stage 4, the agent flags the record for manual legal/finance review instead of sending more emails.

Tone Escalation Matrix (Mandatory Design)

Stage	Trigger	Tone	Key Message	CTA
1st Follow-Up	1–7 days overdue	Warm & Friendly	Gentle reminder, assume oversight	Pay now link / bank details
2nd Follow-Up	8–14 days overdue	Polite but Firm	Payment still pending; request confirmation	Confirm payment date
3rd Follow-Up	15–21 days overdue	Formal & Serious	Escalating concern; mention impact	Respond within 48 hrs
4th Follow-Up	22–30 days overdue	Stern & Urgent	Final reminder before escalation	Pay immediately or call us
Escalation Flag	30+ days overdue	⚠⚠ Flag for Legal	Human review required; no auto email	Assign to finance manager

> ▯ **Email Personalisation:** Each email must include: client name, invoice number, amount due, due date, number of days overdue, and a dynamic payment link or contact detail. The LLM must NOT generate a generic email — all fields must be populated from the data source.

Sample Email Progression

Stage 1 — Warm

Subject: Quick Reminder – Invoice #INV-2024-001 | ▯45,000 Due

Hi Rajesh, I hope you're doing well! This is a friendly reminder that Invoice #INV-2024-001 for ▯45,000 was due on 20 Apr 2025. If you have already processed this, please disregard. Otherwise, you can use the payment link below. Thank you!

Stage 3 — Formal

Subject: IMPORTANT: Outstanding Payment – Invoice #INV-2024-001 (15 Days Overdue)

Dear Mr. Kapoor, Despite our previous reminders, Invoice #INV-2024-001 (▯45,000) remains unpaid as of today, now 15 days overdue. We request your immediate attention. Continued non-payment may impact your credit terms. Please respond within 48 hours.

Stage 4 — Stern

Subject: FINAL NOTICE – Invoice #INV-2024-001 – Immediate Action Required

Dear Mr. Kapoor, This is our final reminder. Invoice #INV-2024-001 (▯45,000) is now 28 days overdue. Failure to remit payment within 24 hours will result in escalation to our legal and recovery team. Please act immediately.

Suggested Tech Stack

Layer	Suggested Options
LLM	GPT-4o / Claude 3.5 Sonnet / Gemini 1.5 Flash / Llama 3 (local)
Agent Framework	LangChain / LangGraph / CrewAI / AutoGen
Data Source	CSV / Excel (pandas) / SQLite / Google Sheets API
Email Send	SMTP (smtplib) / SendGrid API / Mailgun / Dry-run log
Scheduling	APScheduler / Celery + Redis / GitHub Actions cron
Logging	SQLite audit table / JSON log file / Google Sheets
UI (Optional)	Streamlit dashboard showing queue, sent, escalated counts

COMMON REQUIREMENTS (Applicable to Both Tasks)

1. Mandatory Technical Disclosures

Every submission must include a dedicated **Technical Stack & Decision Log** section (in README or a separate doc) covering the following:

Disclosure	What to Include
LLM Chosen	Model name, provider, version (e.g. GPT-4o-2024-08-06). Justify why this model over alternatives — cost, context window, tool-calling support, etc.
Agent Framework	Framework name & version. Explain the architecture — ReAct loop, plan-and-execute, multi-agent, etc. Include an agent flow diagram.
Prompt Design	Share key system prompts. Explain how you structured them and what guardrails you applied.
Security Mitigations	See Section 2 below — this is a graded component.

2. Security Risk Mitigation

⚠ This section is **mandatory** and will be assessed. Students who skip security documentation will receive zero marks for this component regardless of prototype quality.

Risk	Description	Mitigation Strategy
Prompt Injection	Malicious input manipulating agent behaviour	Input sanitisation, structured output schemas, output parsers with validation
Data Privacy / PII	Resume/email data contains personal info	Local processing, data masking in logs, no plaintext PII in prompts sent to cloud LLM where possible
API Key Exposure	LLM/email API keys leaked in code	Use .env + python-dotenv; never hardcode; add .env to .gitignore; use secrets manager in prod
Hallucination Risk	LLM generating false scores or wrong email content	Structured output (JSON mode / Pydantic), confidence thresholds, human-in-the-loop review step
Unauthorised Access	Anyone triggering the agent endpoint	API key / OAuth authentication on any exposed endpoint; rate limiting
Email Spoofing	Emails appearing from wrong sender (Task 2)	SPF/DKIM/DMARC setup; use verified sender domain; dry-run mode in testing

4. Deliverables & Submission Guidelines

- **GitHub Repository** (public or shared link) with all source code, .env.example, requirements.txt, and README.md.
- **README.md** must include: Project overview, setup instructions, agent architecture diagram, LLM & framework choice with rationale, security mitigations.
- **Demo:** A 3–5 minute screen recording OR live demo showing the agent running end-to-end on sample data.
- **Presentation Deck:** 8–10 slide deck summarising problem, solution, architecture, results, and learnings.
- **Sample Output:** Include at least one sample output (shortlist report PDF for Task 1 / sample emails log for Task 2).

Pro Tips from the Team

- Start with a working end-to-end skeleton first — even with mock data — then add intelligence layer by layer.
- Use structured output (JSON mode or Pydantic models) from day one. It will save you hours of debugging parsing errors.
- Document your prompt iterations. Mentors want to see your thought process, not just the final prompt.
- For Task 1: Test with at least 5 resumes (mix of good match, partial match, no match) to validate your rubric.
- For Task 2: Always implement a dry-run / sandbox mode so you don't accidentally email real clients during testing.
- LLM costs add up. Use caching (e.g. LangChain SQLite cache) during development to reduce API calls.
- Observability matters: Consider adding LangSmith or Langfuse tracing for bonus marks.